

This course has already ended.

« Round 16 (https://plus.cs.aalto.fi/studio_2/k2022/...

Chapter 16.1: Files and chess » (<https://plus.cs.aalto....>

CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 16 (https://plus.cs.aalto.fi/studio_2/k2022/w16/)

/ Chapter 16.0: Software Testing

Chapter 16.0: Software Testing

About this page:

Key questions Why are programs tested? How can the functionality of the program be tested manually and automatically?

Key questions Unit testing.

What will I do? Mostly reading

Indicative difficulty level: Not very challenging.

Rough estimate of workload:? 1 hour

Point value: S10.

Related projects: None

"Program testing can be used to show the presence of bugs, but never to show their absence!"

Edsger Dijkstra

Program testing is carried out to examine empirically the quality and properties of the program. The main aspect that is considered is correct execution of the program, and the most important task is to find errors in the program execution so that these errors can be corrected. Other potential aspects considered in testing include program performance and its usability. However, in this course we do not consider them.

Because any bigger program often has an unlimited number of input values and possible internal states, testing can not prove that the program is completely flawless. Testing intends to demonstrate that the program works as well as possible in accordance with its *specification*. If it does not, testing facilitates identifying the location of errors in program code. The terms *error* and *bug*, therefore, express a deviation from the specification.

Good program development process typically generates, in addition to the actual source code, a collection of carefully selected test cases. These test the program correctness in basic cases, like the most typical input values and most commonly used scenarios, as well as at the limits of the specification and in special cases, such as what happens with incorrect inputs.

Unit testing

Unit testing is a software testing method in which the function of individual program parts is tested, if possible, separately from the other code in order to establish that the tested parts meet their requirements. Unit testing is typically performed using some unit testing libraries that perform the testing and compilation of the results. Libraries usually also work with IDEs, making it easy and quick to run tests while programming. For larger projects, unit tests are automatically performed whenever new code is imported into the project. The advantages of unit testing include the automation provided by the libraries and the reassurance that well unit-tested classes will work correctly.

Unit test

A unit test is in practice a tiny program, often named in a special way or an "annotated" method that begins by initializing the tested part to some initial state, in which the test is to be performed. If, for example, we were testing the movie database that we managed in the fall, it would be a part of the test to start the program with the desired text file. Equally, the beginning of a test may be to set up classes so that the tested code can be tested separately. Here we get help from the substitute classes mentioned in the chapter 14.

After the program has been set to a suitable initial state, we perform the planned tests and check if the expected change in the program state occurred. This check is done in almost every unit test library using the *assert* method or macro. Here we briefly introduce the *ScalaTest* library for testing. *ScalaTest* offers a number of different approaches to testing of which we present the *AnyFlatSpec* test class that is similar to conventional unit testing. First, we show a code snippet, after which it is easier to explain further.

```
import collection.mutable.Buffer
import org.scalatest.flatspec.AnyFlatSpec
import org.scalatest.matchers.should.Matchers

class BufferTest extends AnyFlatSpec with Matchers {

  "Buffer" should "store every appended value as its last element." in {
    val myBuffer = Buffer[String]()
    assume(myBuffer.isEmpty)

    myBuffer += "Cat"
    assert("Cat" === myBuffer.last)

    myBuffer += "Dog"
    assertResult("Dog") {
      myBuffer.last
    }
  }

  it should "return None from lastOption when it is empty." in {
    assert(Buffer[String]().lastOption === None)
  }
}
```

It is good to give a unit test a name, which allows for easy interpretation of the results without re-reading the code. In this test type, we first name the item to test, and the string after that describes the expected action that is tested.

Between these strings, there are special words used by the test library - in reality, these are methods whose parameters are the descriptions previously mentioned and the test itself.

Each test contains some kind of assert statement. These are usually comparison expressions that must be true to pass the test. One test method may have multiple assertions.

To enable the test library to recognize that this is a test, the class must be inherited from the test class in the test library.

The Triple equality symbol is also a specialty of the library. Where the normal comparison operator `==` yields a truth value, but forgets the values that are compared, the test library's own operator remembers these values. This is quite handy when it comes to finding out what went wrong.

`assertResult` is better than simply `assert` and `===` because it clearly distinguishes the value expected by the test and the realized value.

In our example, the class `BufferTest` is a collection of small tests. `should` is a method that uses Scala's features in a versatile way. The end result is a so-called *domain-specific language*, DSL, which is, in this case, a special notation for writing tests, but at the same time, it is valid Scala. How this trick is performed is not discussed here. The essence of the "should" line is a clear verbal description of the test case. In addition to this, any more complicated test is worth commenting well - such information can be of great benefit if the test occasionally catches an error. For more information, you can read ScalaTest's own test tutorial (http://www.scalatest.org/user_guide/writing_your_first_test).

The method `assert` takes as a parameter a Boolean expression, and if the value of the expression is true, this part of the test is accepted. Similarly, the method `assertResult` takes two parameter lists, the first of which is the expected value and the other one is the value returned by the test. `assertResult` will thus be able to show which test failed, what value was expected and which one was obtained instead. The assertions use *exceptions* for transferring the failed test data to the test library. Exceptions were discussed more in Chapter 15.2. For more information about ScalaTest assertions, see here (http://www.scalatest.org/user_guide/using_assertions).

The test also had a method `assume`, which allows the test to be skipped if a precondition is not true. In our first test, we used it to make sure that the initial state of the test (buffer was empty) was certainly true before we started testing the buffer.

There is also a method `fail` with which a test can be unconditionally failed.

Test environment

The previous chapter introduced the auxiliary classes `Dummy`, `Stub`, and `Mock`, which are used to help in testing and implementing a program. Using them instead of the actual classes enables these classes to perform during the test exactly in the same way as in the specification. This is particularly useful if the actual classes have not been implemented yet or have significant side effects and dependencies. The ability to replace, for example a class that maintains a connection to a printer, with an auxiliary class allows running tests that would be burdensome or even expensive to execute automatically. In addition, this way it is easy to create a special situation in which our `Mock`, for example, simulates the situation where the disk space is full, by answering the method calls as desired.

Test coverage

Testing should not be restricted to a few of the easiest cases which come to one's mind. Rather, one should act systematically and pay attention to the *coverage* of the test set. Coverage can be evaluated by several different measures associated with the code and its execution, such as:

- *Function coverage*: Have all methods been called?
- *Statement coverage* or *line coverage*: Has every program statement or line been executed?
- *Branch coverage*: Is every branch of every conditional statement (or equivalent) executed?
- *Condition/decision coverage*: Is every conditional expression evaluated both as true and false?
- *Path coverage*: Has every possible execution path in the program been executed. This is usually not possible to achieve.

There are ready-made tools to evaluate these coverage metrics that can be used in testing, which report, for example, how many percent of the statements in the program the tests execute. Possibly, they also show statements which are not executed in the tests.

Unfortunately, even achieving 100% coverage of these coverage measures (except for the path coverage, which generally cannot be reached in tests) is not always enough to produce a comprehensive set of tests for the program, as the following example will show:

Assignment

The **ratio** method should calculate the ratio of two integers given as parameters as accurately as possible and return, if possible, this value as a floating-point number wrapped in an `Option` wrap. The ratio should be calculated so that the larger parameter is divided by the smaller one. Special situations are handled with the `Option` class instead of exceptions.

The implementation below does not meet the specification. Think about how the code should work in normal and abnormal situations?

```
/**
 * Calculates the exact ratio of the given integers (ratio of greater to smaller), if possible. Returns the
 * result of the calculation as a Double wrapped in an Option. In case of a divide-by-zero attempt returns None.
 */

def ratio(n: Int, m: Int): Option[Double] = {
  if (n < m)
    Some(m/n)
  else
    Some(n/m)
}
```

Consider that our test set includes the following tests:

```
assert(ratio(2,4) === Some(2))

assert(ratio(4,2) === Some(2))
```

It may be easy to end up with such a test set if you are going to build tests based on your existing implementation. The set provides 100% function coverage, statement coverage, branching coverage, and condition coverage. However, something essential has been missed, as our program does not meet its requirements even though it passes both tests. Coverage alone is not enough to assess the quality of the test. But this test set hides the fact that we performed an integer division instead of a Double division.

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 1

This course has been archived (Thursday, 1 June 2023, 12:00).

Question 1 10 / 10

Show anyway

How could you improve tests? Choose from below the smallest number of tests (3) which should have been among our test set instead of the given ones.

Note that the implementation of method `ratio`, given above does not match the specification "calculate the exact ratio of two integers given as parameters as accurately as possible. The ratio should be calculated so that the larger parameter is divided by the smaller one."

The specification is not exact, so you may need to do some interpretation. In practice, it is good to write down possible interpretations incorporated with the test, so that if the test fails, we can distinguish between interpretation error and programming error.

- ☐ `ratio(1,4) == Some(0)`
- ☐ `ratio(1,4) == Some(0.25)`
- ☐ `ratio(1,4) == Some(4)`
- ☐ `ratio(1,4) == None`
- ☐ `ratio(0,4) == Some(0)`
- ☐ `ratio(0,4) == Some(Double.PositiveInfinity)`
- ☐ `ratio(0,4) == Some(4)`
- ☒ **`ratio(0,4) == None`**
- ☒ **`ratio(2,3) == Some(1.5)`**
- ☐ `ratio(2,3) == Some(0.66)`
- ☒ **`ratio(3,2) == Some(1.5)`**
- ☐ `ratio(3,2) == Some(0.66)`
- ✓ Correct!

Submit

Therefore, when considering the coverage, one should not only be content with the achievement of 100% of the coverage dimensions (which, of course, should be pursued), but one should also consider the specification of the program and seek to find relevant test cases.

Testing is not always easy and straightforward. The program itself may already be difficult to test, but much depends, for example, on the structure of the program, e.g., how easy it is to separate parts of the program for testing. In many cases, the internal state of an object depends on the execution history, whereby a single test may pass, but successive execution of multiple operations may fail. For example, creating a test case that would catch an error which has been observed when using a program may be difficult, because errors can be repeated occasionally and possibly very rarely.

Testing Tips

Thus, the code coverage does not guarantee that the code would work correctly, but only that every row in the erroneous code has been visited. However, the end result of the test may depend, for example, on the first line that the test executed, even if the coverage measure appears to have completed all rows. How should test cases be selected? For example, testing a list:

Write multiple tests and have multiple asserts in each test

For common cases, you should write more than one test. In our example: Put the numbers 1-10 in the buffer and check that the numbers really are there. (positive test)

```
import collection.mutable.Buffer
import org.scalatest.flatspec.AnyFlatSpec
import org.scalatest.matchers.should.Matchers

class ListTest extends AnyFlatSpec with Matchers{

  "Buffer.contains" should "find all values the Buffer." in {
    val buffer = Buffer[Int]()

    for (i <- 1 to 10) {
      buffer += i
    }

    for (i <- 1 to 10) {
      assert(buffer.contains(i))
    }
  }

  it should "not find items that are not in the Buffer." in { ...
    //Next test
```

Test outside the boundaries

Another common case: Search from a buffer of items an item that is not there and check that the return value is false. (negative test)

```
it should "not find items that are not in the Buffer." in {
  val buffer = new Buffer[Int]

  for (i <- 1 to 10) {
    buffer += i
  }

  assert(!buffer.contains(0))
  assert(!buffer.contains(11))
}
```

Test values that are just inside boundaries

It is good practice to write the tests at the limits of permissible values in the specification - values that are just at the boundary of the activity defined in the specification.

For example, an empty buffer and a single-item buffer are such. In our example, an item is removed from the buffer when there is only one element. Everything should work without an exception.

```
"Buffer.remove" should "remove the only item in a buffer resulting in an empty Buffer." in {  
    val buffer = Buffer[Int](50)  
  
    buffer.remove(0, 1)  
    assert(!buffer.contains(50))  
    assert(buffer.isEmpty)  
}
```

Test that exceptions are thrown and errors are properly handled

It is also important to test how the program behaves in error and special cases.

In our example, we try to remove an element in a negative index location and check that the code results an exception as in the specification.

```
"Buffer.remove" should "throw the appropriate exception on negative indices" in {  
    val buffer = Buffer[Int](50)  
  
    intercept[IllegalArgumentException] {  
        buffer.remove(-1,1)  
    }  
}
```

Other Tips

Remember, too, that if you use randomness when you create test data, initialize the random number generator with a constant value.

- The error situations can then be repeated and debugged.
- Or save the seed number you are using so that you can repeat the tests.

Easier testing

Some of the things that make testing easier are also common good coding practices. When class and trait interfaces are, where possible, simple and clear, and there are few dependencies between classes, they are easier to test. It is advisable to split large classes into several classes, with the aim of distinguishing different functionalities, for example, into their own traits. After that, each trait can be tested separately, if necessary.

In the previous chapter, a few "substitute classes" were briefly introduced. There is much use for them in unit testing, since virtually all classes need other classes to implement their own functionality. For a complex interface, it is difficult to write auxiliary classes simulating real execution.

In addition, if a class directly invokes the constructors of the classes it uses, then replacing the dependencies is challenging. If dependencies are passed on to a class constructor, then there is no problem. Interested people can read more about the subject, for example, in Wikipedia. Dependency injection (https://en.wikipedia.org/wiki/Dependency_injection).

Similarly, the dependencies that emerge through inheritance are more difficult to handle than those that emerge from the references.

Test-Driven Development (TDD)

Test-driven development is a testing-enhancing software development process where everything starts from writing tests. As soon as the class's external interface is known, tests for the class activity are written. The function will only be implemented when the tests are in place.

Correspondingly, when errors are found in the code, a correction is initiated by writing a test indicating the existence of the error and only then start correcting the error. The ability to test the error automatically facilitates the repair process and later on the test prevents the error emerging again. Test-Driven-Development is described in more detail, for example, in Wikipedia (http://en.wikipedia.org/wiki/Test-driven_development)

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Time spent: (*) Pakollinen Please estimate the total number of minutes you spent on this chapter (reading, assignments, etc.). You don't have to be exact, but if you can produce an estimate to within 15 minutes or half an hour, that would be great.

17

Show anyway

"I feel that I have understood the most important things in this chapter." (*) Pakollinen

- ☒ fully agree
- ☐ somewhat agree
- ☐ somewhat disagree
- ☐ fully disagree
- ☐ I'm unable to answer or don't want to comment.

Submit an update

« Round 16 (https://plus.cs.aalto.fi/studio_2/k2022/...

Chapter 16.1: Files and chess » (<https://plus.cs.aalto....>